第 04 章 Mapping 入门：字段类型与动态映射

很多检索效果问题，表面看是”查询写得不对”，本质却是”字段类型建错了”。Mapping 是 ES 的数据契约，一旦错误，修复成本通常高于一开始多花的建模时间。

本章目标

- 掌握字段类型与查询能力的关系
- 学会控制动态映射带来的风险

---

1. Mapping 的核心价值与风险认知

1.1 Mapping 作为数据契约的本质

Mapping 在 Elasticsearch 中扮演着**数据契约**的核心角色，它定义了文档中每个字段的数据类型、索引方式以及可支持的查询能力。这一契约一旦确立，便直接决定了后续所有数据操作的边界与可能性。字段类型的选择绝非简单的技术细节，而是从根本上塑造了索引的存储结构、查询性能和功能完备性。

从底层实现来看，Mapping 控制着三个核心索引结构的构建策略：

索引结构	功能说明	受 Mapping 类型影响
倒排索引（Inverted Index）	词项到文档列表的映射，支撑全文搜索	`text` 类型经分词后构建，`keyword` 直接索引完整值
正排索引（Doc Values）	文档到字段值的列式存储，支撑排序和聚合	`keyword`、`numeric`、`date` 等类型默认启用，`text` 默认禁用
点数据（Points）	多维数值索引，支撑范围查询和地理搜索	`date`、`geo_point`、`numeric` 等类型构建 BKD 树索引

字段类型与查询能力的刚性绑定是 Elasticsearch 架构的核心特征。`text` 类型经过分析器（Analyzer）的分词处理，将连续文本拆分为独立词条，从而实现模糊匹配和相关性评分，但代价是**无法支持排序、聚合和精确匹配**——原始字符串信息在分词过程中已经丢失。反之，`keyword` 类型保持原始值的完整性，直接建立正排索引，能够高效支持精确过滤、排序和聚合，但**丧失了全文搜索的灵活性**。

这种设计上的权衡意味着：Mapping 错误不会在写入阶段暴露，而会在查询阶段以”静默失败”的形式呈现——查询语法正确、执行无异常，但结果与业务预期严重偏离。更为严峻的是，Elasticsearch 的 Mapping 具有**不可变性**——一旦字段被创建并写入数据，其类型便无法直接修改。修复错误的唯一途径是创建新索引、重新定义 Mapping、执行 Reindex 数据迁移、切换别名指向，这一完整流程在 TB 级数据规模下可能持续数小时甚至数天，期间需要精细的容量规划、进度监控和失败回滚机制。

1.2 常见问题的根源分析

检索效果问题的表象与本质之间存在深刻的认知错位。开发者和运维人员面对”搜索结果不准确”、“排序结果异常”或”聚合数据错误”时，往往习惯性地将排查焦点放在查询语句的优化上——调整布尔逻辑、修改评分参数、尝试不同的查询类型——却忽视了**超过 60% 的检索效果问题根源在于 Mapping 设计阶段**。

典型的问题模式包括：

问题现象	根本原因	错误 Mapping 配置	正确方案
精确匹配用户 ID 失败	`text` 类型分词导致原始值丢失	`"user_id": { "type": "text" }`	`"type": "keyword"`
按价格区间筛选无结果	`keyword` 类型字符串比较而非数值比较	`"price": { "type": "keyword" }`	`"type": "scaled_float", "scaling_factor": 100`
时间范围查询返回异常	`date` 格式解析失败或时区混乱	未指定 `format` 或格式不匹配	显式配置 `format` 并统一 UTC 存储
对象数组查询交叉匹配	`object` 类型扁平化破坏元素关联	`"comments": { "type": "object" }`	`"type": "nested"`
聚合统计结果无意义	`text` 类型词条分布而非原始值分布	对 `text` 字段执行 `terms` 聚合	使用 `keyword` 子字段或改用 `keyword` 类型

生产环境变更 Mapping 的连锁反应进一步放大了设计失误的代价。在微服务架构中，Mapping 变更可能触发跨服务的数据契约调整：上游数据生产者的字段格式变更、下游消费者的查询逻辑适配、中间消息队列的 Schema 演进，形成复杂的依赖网络。更为隐蔽的风险是**历史数据的兼容性**——若错误 Mapping 已导致脏数据写入（如数值字段被截断、日期格式被错误解析），数据清洗和一致性校验将成为额外的沉重负担。

---

2. 核心字段类型详解与选型策略

2.1 字符串类型：text 与 keyword 的深层差异

字符串类型是 Elasticsearch 中最常用也最复杂的类型体系。自 5.x 版本移除统一的 `string` 类型后，`text` 与 `keyword` 的分化体现了搜索引擎对”全文检索”与”结构化数据”两种 fundamentally different 访问模式的深刻洞察。

2.1.1 text 类型：全文搜索的基石

`text` 类型的核心机制在于**分析管道（Analysis Pipeline）**的三阶段处理：

1. **字符过滤器（Character Filter）**：预处理原始文本，如去除 HTML 标签、替换特定字符
2. **分词器（Tokenizer）**：将连续文本切分为独立词条，如按空白和标点分割
3. **词条过滤器（Token Filter）**：后处理词条，如小写转换、停用词移除、同义词扩展

以中文场景为例，使用 IK 分词器的 `text` 字段会将”高性能搜索引擎”处理为 [“高性能”、“搜索”、“引擎”] 三个词条，每个词条都在倒排索引中建立指向原文档的映射。这种**词项级别的索引结构**使模糊匹配成为可能——用户搜索”搜索引擎”时，即使词序与原文不同，仍能通过词条交集找到相关文档，并基于 TF-IDF 或 BM25 算法计算相关性评分。

`text` 类型的典型应用场景高度聚焦于**内容密集型全文检索**：

- 文章内容与新闻资讯的主体文本搜索
- 电子商务平台的商品描述和详情页检索
- 用户评论、社交媒体帖子的情感分析与关键词提取
- 日志消息的自由文本过滤与异常检测

然而，分词机制带来的灵活性以**明确的能力边界**为代价。由于原始文本在索引阶段被拆解为离散词条，`text` 字段无法直接支持以下操作：

操作类型	失败原因	错误尝试	系统响应
精确匹配	原始值信息丢失	`term` 查询完整字符串	无匹配或意外匹配
排序	无单一可比值	`sort` 按 `text` 字段	执行失败或强制启用 fielddata

Table 3 – continued

操作类型	失败原因	错误尝试	系统响应
聚合	词条分布无业务意义	terms 聚合 text 字段	返回分词后的词条统计

强制启用 fielddata 以支持这些操作是生产环境的**反模式**——它会将倒排索引加载到 JVM 堆内存，对大规模数据极易引发 OutOfMemory 错误，且性能远低于原生支持这些操作的类型。

2.1.2 keyword 类型：结构化数据的支柱 与 text 类型的"拆解"哲学相反，keyword 类型坚持**原始值的完整性**。字段值不经任何分词处理，直接作为单一词条存入倒排索引，并同步构建正排索引（Doc Values）以支持高效聚合与排序。

keyword 类型的核心能力矩阵涵盖四个维度：

能力	实现机制	典型查询/操作	性能特征
精确匹配过滤	倒排索引的直接查找	term、terms、prefix 查询	O(1) 哈希查找
范围查询	字符串字典序比较	range 查询（gt/gte/lt/lte）	基于排序索引的区间扫描
排序	正排索引的列式遍历	sort 参数	内存高效，支持多字段排序
聚合分析	正排索引的分组统计	terms、cardinality、composite 聚合	基数敏感，高基数字段需优化

keyword 类型的典型应用场景具有鲜明的结构化特征：

- **标识符系统**：用户 ID、订单编号、商品 SKU、会话 ID——需要精确等价比较
- **枚举状态**：订单状态（待支付/已发货/已完成）、审批结果（通过/驳回/待审）——需要分组统计
- **标签体系**：商品分类、文章标签、用户画像标签——需要多选过滤和标签云聚合
- **标准化编码**：邮箱地址、主机名、国家代码、币种代码——需要格式一致性保证

keyword 类型的关键配置参数 ignore_above 值得特别关注。该参数指定超过长度的字符串将被忽略索引（默认为 256 字符），这一设计旨在防止超长字符串（如 URL、堆栈跟踪）对索引结构和内存使用造成不成比例的影响。对于可能超出此限制的场景，需要评估是否真的需要精确匹配能力，或考虑截断存储、哈希摘要等替代方案。

2.1.3 多字段（multi-fields）设计模式 生产环境中的复杂业务需求往往要求同一数据源同时支持全文搜索和结构化操作——商品名称既要能被关键词模糊搜索，又要能按名称精确排序和统计热门商品。Elasticsearch 的**多字段（multi-fields）**机制正是为解决这一矛盾而生。

多字段允许为单个字段定义多个子字段，每个子字段独立配置类型和分析器，从同一原始数据构建不同的索引结构。最典型的模式是将主字段设为 text 类型用于全文搜索，同时创建 keyword 子字段用于排序、聚合和精确匹配：

```
{
  "mappings": {
    "properties": {
      "product_name": {
        "type": "text",
        "analyzer": "ik_max_word",
        "fields": {
          "keyword": {
            "type": "keyword",
            "ignore_above": 256
          }
        }
```

```
      }
    }
  }
}
```

此配置创建了两个逻辑字段：

- `product_name`：`text` 类型，IK 分词器，支持”智能手机”、“手机智能”等模糊匹配
- `product_name.keyword`：`keyword` 类型，支持精确匹配、按名称字母排序、`terms` 聚合统计

查询时根据意图选择字段路径：

- 全文搜索：`"match": { "product_name": " 智能手机" }`
- 精确过滤：`"term": { "product_name.keyword": "iPhone 15 Pro" }`
- 排序聚合：`"sort": [{ "product_name.keyword": "asc" }]`、`"aggs": { "by_name": { "terms": { "field": "product_name.keyword" } } }`

多字段设计的成本在于**存储开销的倍增**——同一数据被以两种结构分别索引，索引大小通常增加 20%-50%。但这一代价在功能完整性和查询灵活性面前几乎总是值得的。事实上，Elasticsearch 8.x 的动态映射对字符串字段的默认行为即为此配置，体现了其作为生产标准的地位。

2.2 数值类型体系

Elasticsearch 提供了精细化的数值类型家族，以在存储效率、取值范围和计算精度之间取得最优平衡。

2.2.1 整数类型家族

类型	存储空间	取值范围	典型应用场景	选型建议
`byte`	1 字节	-128 ~ 127	优先级编码（1-5）、百分比值、小型枚举	明确受限场景的最优选择
`short`	2 字节	-32,768 ~ 32,767	页面序号、小时级时间偏移、中小型计数器	预留 10 倍增长空间
`integer`	4 字节	-2^{31} ~ $2^{31}-1$（约 ±21 亿）	通用计数、用户积分、年龄、年份	大多数业务场景的默认选择
`long`	8 字节	-2^{3} ~ $2^{3}-1$	时间戳（毫秒级）、全局唯一 ID、超大累计值	明确需要大范围的场景

类型选择的核心原则是**在满足需求的前提下选择最小存储类型**。这一优化的依据在于 Lucene 索引的底层实现：更小的字段宽度意味着更紧凑的倒排索引、更快的压缩解压速度、更高的缓存命中率。在亿级文档规模下，合理的类型选择可节省数 GB 存储空间，并显著提升聚合查询的内存效率。

需要特别注意的是，Elasticsearch 的动态映射对 JSON 整数默认推断为 `long` 类型，这种”安全但冗余”的策略对于明确不会超出 `integer` 范围的字段是一种浪费。生产环境应通过显式 Mapping 或动态模板覆盖，将字段映射为更精确的类型。

2.2.2 浮点类型家族

类型	存储空间	精度特性	典型应用场景	关键限制
`float`	4 字节	单精度，约 6-7 位有效数字	科学计算、传感器读数、近似评分、地理坐标	精度损失风险，不适合金融计算
`double`	8 字节	双精度，约 15-16 位有效数字	高精度统计分析、复杂数学运算、科学模拟	仍有浮点误差，非精确小数
`half_float`	2 字节	半精度，约 3-4 位有效数字	机器学习特征向量、对精度极不敏感的近似值	范围受限，需评估溢出风险
`scaled_figure`	8 字节（long 存储）	由 `scaling_factor` 决定的固定精度	**货币金额**、费率百分比、需要精确小数的财务数据	需预设精度因子，范围与精度权衡

`scaled_float` 是金融和电商场景的**关键优化类型**。其工作原理是通过固定的缩放因子将浮点数转换为整数存储，查询时再反向转换。配置 `"scaling_factor": 100` 时，价格 19.99 元存储为整数 1999，既保证了两位小数的精确表示，又享受了整数运算的性能优势和零浮点误差。

选择 `scaling_factor` 时需权衡精度与范围：因子 100 支持两位小数，因子 10000 支持四位小数，但会相应压缩可表示的最大值范围。对于多币种系统，建议统一基础货币单位（如最小货币单位”分”）并配合 `long` 类型，以彻底消除浮点误差风险。

2.3 时间与布尔类型

2.3.1 date 类型 `date` 类型在 Elasticsearch 内部统一转换为 **UTC 时区的毫秒级时间戳（`long` 类型）**存储，这一设计确保了跨时区数据的一致性和可比性。对外暴露时，`format` 参数支持灵活的输入格式解析：

```
{
  "create_time": {
    "type": "date",
    "format": "yyyy-MM-dd HH:mm:ss||yyyy-MM-dd||epoch_millis"
  }
}
```

多格式配置使用 `||` 分隔，系统依次尝试匹配直至成功。这种灵活性提升了数据接入的容错性，但也增加了解析阶段的 CPU 开销。生产环境建议通过 ingest pipeline 统一数据清洗和格式标准化，减少运行时的格式推断不确定性。

`date` 类型的查询能力极为丰富：

- **绝对时间范围**：`"gte": "2024-01-01", "lte": "2024-12-31"`
- **相对时间表达式**：`"gte": "now-7d/d"`（7 天前零点至今）
- **日期数学运算**：`"lt": "now+1M"`（一个月后），`"gte": "2024-03-01||+1M"`（指定日期起一个月后）

聚合方面，`date_histogram` 是时间序列分析的核心工具，支持按固定间隔（`1d`、`1h`、`1w`）或日历间隔（`month`、`quarter`、`year`）分组，配合 `time_zone` 参数可正确处理业务时区的日界统计。

2.3.2 boolean 类型 `boolean` 类型是二元状态的最简洁表达，仅接受 `true`、`false` 以及字符串形式的 `"true"`、`"false"`。其 1 位的实际存储效率使其成为状态标记的理想选择：是否发布、是否启用、是否删除、是否付费等。

设计建议采用统一的命名前缀（如 `is_published`、`is_deleted`、`is_vip`）以提升可读性，并在应用层规范化输入，避免 `"1"`/`"0"` 或 `"yes"`/`"no"` 等模糊表示导致的序列化问题。

2.4 其他核心类型

类型	核心特性	适用场景	关键限制
`binary`	Base64 编码存储，不参与索引	小型缩略图、文档摘要、加密字段	不可搜索，单字段上限 512MB，大文件应使用对象存储
`range` 家族	区间作为单一值存储，支持关系查询	酒店预订时段、价格区间、IP 网段	不支持排序，聚合能力有限

`range` 类型的查询语义包括：`INTERSECTS`（区间有重叠）、`CONTAINS`（包含某点）、`WITHIN`（在某区间内），这些专用操作避免了应用层将区间拆分为起止字段的繁琐处理。

---

3. 复杂数据结构类型

JSON 的嵌套结构是现代数据建模的常态，Elasticsearch 通过 `object`、`nested`、`flattened` 三种机制提供差异化的处理能力，选型错误将直接导致查询结果失真或性能灾难。

3.1 object 类型：层级结构的默认选择

`object` 类型是 Elasticsearch 对 JSON 对象的自然映射，无需显式声明即可使用。其核心机制是**扁平化（Flattening）**：嵌套对象的字段被展开为顶层字段的带点路径。

例如，文档 `{"user": {"name": "Alice", "age": 30}}` 实际存储为 `user.name` 和 `user.age` 两个独立字段。这种扁平化对单层对象查询完全透明，支持标准的 `term` 或 `match` 查询。

然而，扁平化机制在处理**对象数组**时暴露致命缺陷。考虑订单文档中的商品列表：

```
{
  "order_items": [
    {"sku_id": "SKU-A", "price": 100},
    {"sku_id": "SKU-B", "price": 200}
  ]
}
```

`object` 类型将其扁平化为 `order_items.sku_id: ["SKU-A", "SKU-B"]` 和 `order_items.price: [100, 200]` 两个独立数组。此时查询 **“sku_id 为 SKU-A 且 price 为 200 的商品”** 会**错误匹配**该文档——因为两个条件分别在不同的数组中满足，而非同一商品对象内。这种**交叉匹配（Cross-matching）**问题是 `object` 类型的固有局限，无法通过查询优化规避。

`object` 类型的适用场景明确限定于：

- **单层嵌套对象**（如用户基本信息、收货地址）
- **无需精确关联查询的对象数组**（如纯标签列表 `["tag1", "tag2"]`，元素本身无内部结构）

3.2 nested 类型：对象数组的精确关联

3.2.1 核心机制 `nested` 类型通过**独立子文档存储**机制从根本上解决交叉匹配问题。数组中的每个对象被索引为独立的隐藏文档，在 Lucene 层面维护完整的文档边界，通过 `_nested` 路径字段与父文档建立关联。

查询时必须使用专用的 `nested` 查询结构，明确指定 `path` 参数和内部查询逻辑：

```
{
  "query": {
    "nested": {
      "path": "order_items",
      "query": {
        "bool": {
          "must": [
            {"term": {"order_items.sku_id": "SKU-A"}},
            {"term": {"order_items.price": 100}}
          ]
        }
      },
      "inner_hits": {}
    }
  }
}
```

`inner_hits` 参数用于返回匹配的具体嵌套对象，而非仅返回父文档——这对于列表展示场景（如”显示购物车中满足条件的商品”）至关重要。

3.2.2 与 object 的关键对比

维度	object 类型	nested 类型
存储结构	扁平化，同名字段合并为数组	独立子文档，保持对象边界
查询准确性	条件可能跨对象满足，产生假阳性	严格限定于同一对象内，结果精确
查询语法	标准 term/match，无特殊要求	必须使用 nested 查询 + inner_hits
聚合支持	直接聚合，但结果可能无意义	需 nested / reverse_nested 聚合路径
性能特征	轻量，与常规字段无差异	较重，涉及子文档的联合查询
存储开销	较低（无额外结构）	较高（每个对象独立索引）
更新灵活性	整字段替换	支持部分更新（ES 6.x+）

性能差异源于存储结构的复杂度。`nested` 类型的每个对象都创建独立文档，查询时需要额外的 join 操作关联父子文档，复杂查询的性能通常比同等复杂度的普通查询慢 5-10 倍。官方建议单个文档的 `nested` 对象数量控制在 **1000 以内**，层级不宜超过 **2-3 层**。

3.2.3 典型应用场景 `nested` 类型的决策信号是**“多条件必须同时满足同一数组元素”**：

- **电商订单商品列表**：“购买了 iPhone 15 且数量超过 2 件”的订单查询
- **博客文章评论系统**：“用户 Alice 给出的 5 星好评”的精确筛选
- **用户画像标签组**：“兴趣标签’科技’且权重超过 0.8”的复合条件
- **酒店预订房型信息**：“大床房且含早餐且价格区间”的多维度筛选

对于超大规模嵌套数据（如数万条评论），应考虑数据模型重构——将嵌套内容分离为独立索引，通过父文档 ID 建立关联，以换取更优的写入和查询性能。

3.3 flattened 类型：动态键值对的防爆炸方案

3.3.1 设计背景与核心原理 `flattened` 类型（ES 7.3+）是针对**动态键值对数据**的映射爆炸问题的专项解决方案。在日志监控、用户埋点、物联网指标等场景中，数据常携带大量不可预测的标签或属性——不同服务的日志有截然不同的字段集，用户事件埋点的参数结构随 A/B 实验动态变化，设备遥测的指标集合因型号而异。

若使用 `object` 类型处理此类数据，每个新出现的键都会成为独立字段，字段数量可能迅速膨胀至数千甚至数万。这不仅触发 `index.mapping.total_fields.limit`（默认 1000）的硬限制，更致命的是**集群状态（Cluster State）的频繁更新**——每个新字段都需广播至所有节点，单线程同步成为性能瓶颈，集群响应延迟剧增，甚至引发不稳定。

`flattened` 类型的核心创新在于**将整个 JSON 对象作为单一 `keyword` 字段处理**：对象的所有键值对被解析为扁平化的叶子值，建立统一的倒排索引，而**不为每个子字段创建独立映射**。无论对象内部有多少键、多深层级，Mapping 中仅呈现一个 `flattened` 字段定义。

3.3.2 能力边界与权衡 `flattened` 类型的保护以明确的功能边界为代价：

支持能力	实现方式	不支持能力	替代方案
精确键值匹配	`{"term": {"labels.env": "production"}}`	数值范围查询	将数值字段提取为预定义固定字段
键的存在性检查	`{"exists": {"field": "labels.version"}}`	深层嵌套聚合	应用层预处理或限制查询模式
简单 terms 聚合	按完整键值字符串分组	高亮显示	无替代，设计阶段评估需求
前缀/通配符查询	值的字符串匹配	基于数值的排序	无替代，接受字符串字典序

关键限制在于**所有值都被视为 `keyword` 类型处理**。数值比较按字符串字典序进行（“100” < “2”），这与直觉相悖，需要在设计阶段明确评估查询需求是否匹配。

3.3.3 典型应用场景 `flattened` 类型的最佳适用场景具有共同特征：**键的集合高度动态、查询模式简单（主要为精确匹配和存在性检查）、无需复杂数值分析**：

- **容器/云原生日志标签**：Kubernetes Pod 的 `labels`、`annotations`，键名和数量随部署环境变化
- **APM 监控指标维度**：应用性能监控的自定义上下文，不同应用上报不同维度集
- **用户行为事件属性**：埋点系统的扩展参数，业务迭代频繁添加新属性
- **安全审计动态上下文**：安全事件的环境信息，来源多样且不可预测

配置示例：

```
{
  "mappings": {
    "properties": {
      "kubernetes_labels": {
        "type": "flattened",
        "depth_limit": 5,
        "ignore_above": 256
      }
    }
  }
}
```

`depth_limit` 限制嵌套层级防止过度复杂结构，`ignore_above` 截断超长字符串值控制索引大小。

4. 专用数据类型与场景化选型

4.1 地理空间类型

类型	核心能力	数据格式	典型应用
`geo_point`	点坐标存储、距离计算、边界框查询、地理哈希聚合	`{"lat": 39.9042, "lon": 116.4074}` 或 `"39.9042,116.4074"` 或 `[-71.34, 41.12]`	“附近的商家”搜索、配送范围判定、轨迹点存储
`geo_shape`	复杂几何形状（点、线、多边形、集合）、空间关系判断（相交/包含/不相交）	GeoJSON 或 WKT 格式	行政区域归属、配送范围多边形、地理围栏告警

`geo_point` 的底层使用 GeoHash 或 Quadtree 空间索引，确保大规模坐标数据的高效查询。`geo_shape` 基于 R-tree 或 BKD-tree 索引，查询计算复杂度显著高于点数据，应在确需复杂形状支持时选用。

关键注意事项：地理类型**必须在 Mapping 中显式声明**，动态映射无法自动识别地理数据结构。未预定义 `geo_point` 的字段即使接收 `{"lat": x, "lon": y}` 格式的数据，也会被映射为普通 `object`，导致地理查询 API 完全失效。

4.2 网络与安全类型

`ip` 类型专门处理 IPv4 和 IPv6 地址，内部将地址转换为数值形式存储，支持：

- **精确匹配**：单 IP 地址查询
- **CIDR 范围查询**：`{"term": {"client_ip": "192.168.0.0/16"}}`
- **IP 范围聚合**：按网段统计访问分布

典型应用场景包括安全日志的源 IP 分析、访问控制的白名单/黑名单过滤、GeoIP 地理位置关联的前置处理。配置时可通过 `ignore_malformed` 控制无效 IP 的处理策略，通过 `null_value` 指定空值替换。

4.3 分析与统计类型

`token_count` 类型是分析管道的衍生工具，对指定分析器的输出词条进行计数，将词数作为独立数值字段存储。应用场景包括：

- 内容长度筛选（如”查找超过 1000 词的长文章”）
- 文本复杂度分析（词数分布统计）
- 质量评估指标（过短或过长内容的识别）

配置需指定 `analyzer` 参数确保与搜索分词一致，可选 `enable_position_increments` 控制位置增量计数。`token_count` 是静态计数，索引后不会随文档更新自动刷新。

---

5. 动态映射机制与风险控制

5.1 动态映射的工作原理

动态映射（Dynamic Mapping）是 Elasticsearch 降低使用门槛的核心特性。当文档包含 Mapping 中未定义的字段时，系统依据内置规则自动推断字段类型：

JSON 数据类型	dynamic: true 映射结果	dynamic: runtime 映射结果
null	不添加字段	不添加字段
true / false	boolean	boolean
浮点数	float	double
整数	long	long
对象	object	不添加字段
数组	由首个非 null 元素决定	由首个非 null 元素决定
字符串（匹配日期格式）	date	date
字符串（匹配数值格式，需启用 numeric_detection）	float / long	double / long
字符串（默认）	text + keyword 子字段	keyword

字符串的默认双类型映射是动态映射的”安全但冗余”设计：既保证全文搜索能力，又保留精确匹配可能。然而，这一行为对纯结构化数据造成 **50% 以上的索引存储浪费**——若字段明确无需全文搜索，多余的 text 子字段及其倒排索引纯属冗余。

5.2 动态映射的三档控制策略

Elasticsearch 通过 dynamic 参数提供四档控制策略，形成从完全开放到严格管控的连续光谱：

5.2.1 dynamic: true（默认） "dynamic": true 是开箱即用的配置，新字段被自动检测、添加至 Mapping、建立完整索引。优势是极致的灵活性，新字段立即可搜索聚合；风险在于**类型推断可能不符合预期**（如 IP 地址被识别为 text），且**字段数量无节制增长**可能触发映射爆炸。

适用场景：快速原型开发、探索性数据分析、Schema 高度动态且查询模式简单的场景。

5.2.2 dynamic: false "dynamic": false 采取”接受但忽略”的中间策略：新字段数据被写入 _source 保留，但**不建立任何索引结构**——不可搜索、不可聚合、不可排序，仅在文档取回时可见。

这一模式适用于”先存储、后决策”的数据湖场景：保留完整原始数据以备未来分析，但仅对预定义的核心字段建立索引以控制资源消耗。代价是索引空间被不可搜索数据占用，且用户可能误以为数据已索引。

5.2.3 dynamic: strict "dynamic": "strict" 是最严格的控制策略：任何未在 Mapping 中预定义的字段，写入时直接抛出 strict_dynamic_mapping_exception 异常，**拒绝文档入库**。

这一模式强制要求显式建模，适用于需求稳定、变更管控严格的场景：金融核心交易、医疗记录系统、企业 ERP 等。其实施需要配套的数据契约管理机制——字段变更需经过评审、Mapping 更新、应用发布的完整流程。

5.2.4 dynamic: runtime（ES 8.x） "dynamic": "runtime" 是 Elasticsearch 8.x 引入的 **Schema-on-read** 模式。新字段不作为物理索引存储，而是注册为**运行时字段（Runtime Field）**——查询时从 _source 动态解析计算，零存储开销，但查询性能显著低于预建索引。

运行时字段的核心权衡：

维度	运行时字段	索引字段
存储开销	零（仅 _source 原始数据）	倒排索引 + 正排索引
查询性能	较慢（实时解析计算）	快（直接索引查找）
索引速度	快（无额外索引操作）	较慢（需构建索引结构）
灵活性	极高（可随时修改定义）	低（需重建索引变更）
聚合支持	支持（性能受限）	完全支持

Table 12 – continued

维度	运行时字段	索引字段

适用场景：探索性数据分析（字段价值未验证）、极低频查询字段、Schema 频繁演进的过渡期。验证有价值后，可通过 Update Mapping API 将运行时字段提升为物理索引。

5.3 Mapping Explosion（映射爆炸）的成因与危害

Mapping Explosion 是 Elasticsearch 生产环境中最具破坏力的故障模式之一，其本质是**字段数量失控增长导致的集群状态管理危机**。

核心成因：无限制的动态字段生成。典型触发场景包括：

场景	爆炸机制	典型表现
动态日志标签系统	每个服务、每个版本添加新标签键	字段数从数十激增至数千
用户生成内容平台	用户自定义字段或开放属性	不可预测的键空间膨胀
物联网设备数据接入	不同设备型号上报不同指标集	指标名称与维度组合爆炸
过度扁平化的宽表设计	业务字段过度展开	单表 2000+ 字段的”万能”索引

危害表现：集群状态（Cluster State）需要在所有节点间同步，字段数量激增导致：

- 状态更新延迟：单线程串行处理，大状态变更阻塞其他元数据操作
- 网络传输压力：状态体积膨胀，节点间同步带宽消耗剧增
- 内存占用上升：每个节点缓存完整状态，堆内存压力增加
- 新节点加入缓慢：状态传输和恢复时间延长

Elasticsearch 默认设置 `index.mapping.total_fields.limit: 1000` 作为硬上限，超限后新字段写入被拒绝。但这一”硬着陆”保护往往滞后于性能衰减——在达到限制前，集群可能已因状态更新延迟而响应迟缓。

---

6. 生产级 Mapping 设计实践

6.1 显式映射的核心原则

生产环境的 Mapping 设计应遵循三项核心原则：

原则一：核心业务字段必须预定义

所有支撑主要查询、过滤、排序、聚合功能的字段，必须在索引创建时完整定义，包括类型、分析器、格式、索引选项等。这一原则消除动态推断的不确定性，建立团队共识（代码即文档），便于版本控制和变更审查。

原则二：动态映射范围严格受限

根据数据可控性选择合适的 `dynamic` 模式：

- 核心交易数据：`strict`，强制显式建模
- 日志监控数据：`false` 或 `runtime`，控制索引膨胀
- 探索性数据：`runtime`，验证后提升为索引

原则三：扩展机制前置设计

对确实无法预知的动态字段，规划专门的容纳机制：

- 键值对型动态属性：`flattened` 类型
- 结构化动态对象：`nested` 类型的 `{key, value}` 数组
- 完全开放扩展：独立扩展索引，通过关联查询聚合

6.2 动态模板（Dynamic Templates）的精细化控制

动态模板提供了**基于模式匹配的映射规则自动化**，在保留一定灵活性的同时确保类型一致性。

6.2.1 匹配规则体系

匹配条件	匹配目标	典型模式	优先级
`match_mapping_type`	JSON 解析器检测的数据类型	`"string"`、`"long"`、`"object"`	基础过滤
`match`	字段名（通配符 `*` `?`）	`"*_id"`、`"ip_*"`	常用模式
`match_pattern`	字段名（正则表达式）	`"^ip_.*$"`（需显式启用）	复杂模式
`unmatch`	排除字段名模式	`"*_raw"`（与 `match` 配合）	例外处理
`path_match`	完整点分路径	`"user.*.name"`、`"metadata.*"`	层级匹配
`path_unmatch`	排除路径模式	`"user.system.*"`	层级例外

模板按定义顺序匹配，**首个匹配的模板生效**，后续模板被跳过。设计时应将最具体的规则置于前方，通用规则置于后方。

6.2.2 映射动作定义 匹配成功的字段可应用多种映射动作：

动作类型	配置方式	效果
固定类型映射	`"mapping": { "type": "keyword" }`	创建物理索引字段
运行时字段映射	`"runtime": { "type": "ip" }`	创建查询时解析的运行时字段
完整字段配置	`"mapping": { "type": "text", "analyzer": "ik_max_word", ... }`	自定义分析器、格式等完整参数

模板变量提供动态配置能力：

- `{name}`：替换为实际匹配的字段名
- `{dynamic_type}`：替换为检测到的 JSON 数据类型

示例：为不同语言字段自动选择分析器——`"analyzer": "{name}"` 使 `title_en` 使用 `english` 分析器，`title_zh` 使用 `ik_max_word` 分析器。

6.2.3 典型配置模式　模式一：ID 字段强制 keyword

```
{
  "dynamic_templates": [
    {
      "ids_as_keywords": {
        "match": "*_id",
        "mapping": {
          "type": "keyword",
          "ignore_above": 64
        }
      }
    }
  ]
}
```

确保 user_id、order_id、product_id 等标识符字段统一为 keyword 类型，避免动态映射的 text + keyword 冗余。

模式二：布尔标志字段识别

```
{
  "dynamic_templates": [
    {
      "flags_as_booleans": {
        "match": "is_*",
        "mapping": {
          "type": "boolean"
        }
      }
    }
  ]
}
```

自动识别 is_published、is_deleted、is_vip 等命名规范的布尔字段。

模式三：金额字段精确存储

```
{
  "dynamic_templates": [
    {
      "amounts_as_scaled": {
        "match": "amount_*",
        "mapping": {
          "type": "scaled_float",
          "scaling_factor": 100
        }
      }
    }
  ]
}
```

统一处理 amount_total、amount_discount、amount_paid 等金额字段，保证两位小数精度。

模式四：IP 地址类型推断

```
{
  "dynamic_templates": [
    {
      "ips_as_ip_type": {
        "match": "ip_*",
        "runtime": {
          "type": "ip"
        }
      }
    }
  ]
}
```

将 `ip_address`、`ip_source`、`ip_dest` 等字段映射为 `ip` 类型的运行时字段，验证需求后提升为物理索引。

6.3 索引模板（Index Templates）的环境一致性保障

索引模板（Index Templates）将 Mapping、Settings、Aliases 等配置打包为可复用的模板，新索引按匹配模式自动应用。Elasticsearch 8.x 的**可组合模板（Composable Templates）**架构引入分层设计：

层级	功能	复用方式
组件模板（Component Templates）	可复用的配置片段（settings、mappings、aliases）	被多个索引模板引用
索引模板（Index Templates）	组合组件模板，定义索引匹配模式（`index_patterns`）	直接应用于匹配的新索引

环境一致性保障实践：

- 开发、测试、生产环境共享相同的组件模板源
- 通过 `priority` 参数处理模板重叠冲突
- 版本控制所有模板配置，CI/CD 流程自动化部署
- 时间序列数据结合 ILM（Index Lifecycle Management）策略，实现自动化的索引滚动与生命周期管理

6.4 映射膨胀的三层防御体系

防御层级	机制	实施要点	适用场景
第一层：动态策略	`strict` / `false` / `runtime` 的选择	根据数据可控性选择控制强度，核心索引用 `strict`	所有生产索引
第二层：类型优化	`flattened` 类型处理动态结构	对不可预测的键值对数据强制使用 `flattened`	日志标签、监控指标、用户属性
第三层：硬限制	`index.mapping.total_fields` 等参数	设置合理上限（通常 1000-5000），配合监控告警	最后防线，触发拒绝或人工介入

补充保护参数：

- `index.mapping.depth.limit`（默认 20）：对象嵌套深度限制

- `index.mapping.nested_fields.limit`（默认 50）：nested 类型字段数量限制
- `index.mapping.nested_objects.limit`（默认 10000）：单文档 nested 对象数量限制
- `index.mapping.total_fields.ignore_dynamic_beyond_limit`（ES 8.x）：超限后忽略而非拒绝，提供弹性降级

---

7. 完整实战：电商订单 Mapping 设计

7.1 需求分析

电商订单系统的检索与分析需求具有典型代表性：

需求类别	具体场景	技术映射
全文检索	商品名称、规格描述、品牌信息的关键词搜索	text 类型 + IK 分词器，多字段配置 keyword 子字段
精确过滤	订单状态、用户 ID、支付方式、配送地区、时间范围	keyword、date、ip 等类型的 term/range 查询
聚合分析	按商品品类、销售地区、时间段的销售统计；价格区间分布；客单价趋势	keyword 类型的 terms 聚合，date 类型的 date_histogram，scaled_float 类型的 stats/percentiles
嵌套结构	订单商品明细（SKU、名称、单价、数量、优惠）；物流轨迹节点（时间、地点、状态、操作人）	nested 类型，保证元素内部字段关联
动态扩展	不同业务线的自定义属性（B2B 账期信息、跨境清关状态、营销活动标签）	flattened 类型的 ext_attrs 字段，或独立扩展索引

7.2 Mapping 结构实现

```
PUT /orders
{
  "settings": {
    "number_of_shards": 5,
    "number_of_replicas": 1,
    "index.mapping.total_fields.limit": 2000
  },
  "mappings": {
    "dynamic": "strict",
    "dynamic_templates": [
      {
        "ids_as_keywords": {
          "match": "*_id",
          "mapping": {
            "type": "keyword",
            "ignore_above": 64
          }
        }
      },
```

```
    {
      "flags_as_booleans": {
        "match": "is_*",
        "mapping": {
          "type": "boolean"
        }
      }
    },
    {
      "amounts_as_scaled": {
        "match": "amount_*",
        "mapping": {
          "type": "scaled_float",
          "scaling_factor": 100
        }
      }
    },
    {
      "timestamps_as_date": {
        "match": "*_at",
        "mapping": {
          "type": "date",
          "format": "strict_date_optional_time||epoch_millis"
        }
      }
    }
  ],
  "properties": {
    "order_id": {
      "type": "keyword"
    },
    "user_id": {
      "type": "keyword"
    },
    "order_status": {
      "type": "keyword"
    },
    "payment_method": {
      "type": "keyword"
    },
    "created_at": {
      "type": "date",
      "format": "strict_date_optional_time||epoch_millis"
    },
    "paid_at": {
      "type": "date",
      "format": "strict_date_optional_time||epoch_millis"
    },
    "delivered_at": {
      "type": "date",
      "format": "strict_date_optional_time||epoch_millis"
```

```
    },
    "total_amount": {
      "type": "scaled_float",
      "scaling_factor": 100
    },
    "discount_amount": {
      "type": "scaled_float",
      "scaling_factor": 100
    },
    "shipping_fee": {
      "type": "scaled_float",
      "scaling_factor": 100
    },
    "currency": {
      "type": "keyword"
    },
    "product_name": {
      "type": "text",
      "analyzer": "ik_max_word",
      "fields": {
        "keyword": {
          "type": "keyword",
          "ignore_above": 256
        }
      }
    },
    "product_desc": {
      "type": "text",
      "analyzer": "ik_smart"
    },
    "shipping_address": {
      "type": "object",
      "properties": {
        "province": { "type": "keyword" },
        "city": { "type": "keyword" },
        "district": { "type": "keyword" },
        "detail": {
          "type": "text",
          "analyzer": "ik_smart"
        },
        "zipcode": { "type": "keyword" },
        "contact_phone": { "type": "keyword" },
        "contact_name": { "type": "keyword" }
      }
    },
    "order_items": {
      "type": "nested",
      "properties": {
        "sku_id": { "type": "keyword" },
        "sku_name": {
          "type": "text",
```

```
          "analyzer": "ik_max_word",
          "fields": {
            "keyword": {
              "type": "keyword",
              "ignore_above": 256
            }
          }
        },
        "category_path": { "type": "keyword" },
        "brand_name": {
          "type": "text",
          "analyzer": "ik_max_word",
          "fields": {
            "keyword": { "type": "keyword" }
          }
        },
        "quantity": { "type": "integer" },
        "unit_price": {
          "type": "scaled_float",
          "scaling_factor": 100
        },
        "discount_rate": {
          "type": "scaled_float",
          "scaling_factor": 10000
        },
        "item_amount": {
          "type": "scaled_float",
          "scaling_factor": 100
        },
        "is_gift": { "type": "boolean" }
      }
    },
    "logistics": {
      "type": "nested",
      "properties": {
        "company_code": { "type": "keyword" },
        "tracking_no": { "type": "keyword" },
        "status": { "type": "keyword" },
        "events": {
          "type": "nested",
          "properties": {
            "event_time": { "type": "date" },
            "location": { "type": "keyword" },
            "status": { "type": "keyword" },
            "description": { "type": "text" },
            "geo_location": { "type": "geo_point" }
          }
        }
      }
    },
    "promotion_tags": {
```

```
        "type": "keyword"
      },
      "source_channel": {
        "type": "keyword"
      },
      "device_type": {
        "type": "keyword"
      },
      "client_ip": {
        "type": "ip"
      },
      "ext_attrs": {
        "type": "flattened"
      },
      "is_first_order": {
        "type": "boolean"
      },
      "is_rush_order": {
        "type": "boolean"
      },
      "remark": {
        "type": "text",
        "analyzer": "ik_smart"
      }
    }
  }
}
```

关键设计决策解析:

设计点	决策	理由
`dynamic: strict`	强制显式建模	订单数据核心业务，变更需严格管控
`product_name` 多字段	`text` + `keyword`	同时支持全文搜索和精确排序聚合
`order_items` 嵌套	`nested` 类型	保证 SKU、价格、数量的同一商品关联查询
`logistics.events` 双层嵌套	嵌套 `nested`	支持”某物流公司的某次扫描记录”深层关联
`ext_attrs` 扁平化	`flattened` 类型	安全承载业务扩展字段，防止映射爆炸
金额统一 `scaled_float`	`scaling_factor: 100`	保证两位小数精度，零浮点误差
`client_ip` 专用类型	`ip` 类型	支持 CIDR 范围查询和安全分析

7.3 验证与迭代策略

索引创建后的验证流程:

1. **结构验证**: `GET /orders/_mapping` 确认字段类型、子字段、嵌套结构、动态模板符合预期
2. **采样测试**: 写入代表性文档，验证 `_source` 存储和索引行为

```
POST /orders/_doc
{
```

```
  "order_id": "ORD-20240327-001",
  "user_id": "U123456789",
  "order_status": "paid",
  "product_name": "iPhone 15 Pro 256GB",
  "total_amount": 9999.00,
  "created_at": "2024-03-27T10:30:00+08:00",
  "order_items": [
    {
      "sku_id": "SKU-IPHONE15P-256-DEEP",
      "sku_name": "iPhone 15 Pro 256GB 深空黑",
      "quantity": 1,
      "unit_price": 9999.00,
      "is_gift": false
    }
  ],
  "ext_attrs": {
    "campaign": "spring_sale_2024",
    "channel": "app_store",
    "coupon_code": "SAVE500"
  }
}
```

3. **查询验证**：执行典型查询确认行为正确

```
// 全文搜索 + 状态过滤
GET /orders/_search
{
  "query": {
    "bool": {
      "must": [
        { "match": { "product_name": "iPhone" } },
        { "term": { "order_status": "paid" } }
      ]
    }
  }
}

// 嵌套查询：购买特定 SKU 且单价超过阈值
GET /orders/_search
{
  "query": {
    "nested": {
      "path": "order_items",
      "query": {
        "bool": {
          "must": [
            { "term": { "order_items.sku_id": "SKU-IPHONE15P-256-DEEP" } },
            { "range": { "order_items.unit_price": { "gte": 5000 } } }
          ]
        }
      },
```

```
      "inner_hits": {}
    }
  }
}

// 聚合分析：按日期统计销售额趋势
GET /orders/_search
{
  "size": 0,
  "aggs": {
    "daily_sales": {
      "date_histogram": {
        "field": "created_at",
        "calendar_interval": "day"
      },
      "aggs": {
        "total_revenue": {
          "sum": { "field": "total_amount" }
        }
      }
    }
  }
}
```

4. **性能基准**：使用真实数据量进行压力测试，评估索引吞吐量、查询延迟、聚合响应时间

迭代优化流程：

- 基于查询日志识别慢查询，通过 Profile API 分析执行计划
- 针对性调整 Mapping（如添加 `eager_global_ordinals` 优化高基数字段聚合）
- 重大变更遵循"新建索引 → Reindex 迁移 → 别名切换"的标准流程

---

8. 关键对比与决策速查

8.1 text vs keyword 选型决策树

```
字符串字段需要全文搜索（分词、相关性评分）?
├── 否 → 使用 keyword 类型（或禁用索引）
│    └── 需要排序/聚合/精确匹配? 是 → 纯 keyword; 否 → index: false
└── 是 → 需要排序/聚合/精确匹配同一字段?
     ├── 否 → 使用纯 text 类型（考虑优化 index_options）
     └── 是 → 使用 multi-fields: text 主字段 + keyword 子字段
              全文搜索用 text，排序聚合用 .keyword
```

8.2 object vs nested vs flattened 场景矩阵

数据结构特征	查询需求	推荐类型	关键判断依据
单层对象，无数组	独立字段查询	`object`	默认选择，零额外开销

Table 20 – continued

数据结构特征	查询需求	推荐类型	关键判断依据
对象数组，需元素内多字段联合条件	精确关联查询	nested	必须保持字段关联性，接受性能开销
对象数组，仅需单字段条件	独立元素查询	object	性能优先，接受交叉匹配风险
动态键值对，键名不可枚举	简单键值过滤	flattened	防止映射爆炸，牺牲复杂查询
动态键值对，需数值范围/聚合	复杂分析查询	nested + {key, value} 结构	保留类型信息，增加建模复杂度
深层嵌套（>3 层）	任意查询	扁平化重构	嵌套深度严重影响性能和可维护性

8.3 动态映射策略选择对照表

数据可控性	查询性能要求	变更频率	推荐策略	典型场景
高（Schema 稳定）	高	低	strict + 显式 Mapping	金融核心交易、订单主数据、用户主数据
中（偶有扩展）	高	中	false + 核心字段显式	监控指标索引、配置数据
低（高度动态）	中	高	runtime	探索性分析、实验性特征
低（键值对结构）	中	高	flattened 类型	日志标签、用户画像动态属性
极低（完全开放）	低	极高	runtime + 定期评估提升	临时数据分析、POC 验证

---

9. 实战代码参考

9.1 curl 命令集

创建带动态模板的索引

```
# 设置环境变量
export ES_URL="http://localhost:9200"
export ES_USER="elastic"
export ES_PASS="your-password"

# 创建索引
curl -u "$ES_USER:$ES_PASS" -X PUT "$ES_URL/mapping_demo" \
  -H "Content-Type: application/json" \
  -d '{
    "settings": {
      "number_of_shards": 1,
      "number_of_replicas": 0
    },
    "mappings": {
      "dynamic": "strict",
      "dynamic_templates": [
        {
```

```
        "ids": {
          "match": "*_id",
          "mapping": {
            "type": "keyword"
          }
        }
      },
      {
        "strings_as_keywords": {
          "match_mapping_type": "string",
          "unmatch": "*_text",
          "mapping": {
            "type": "keyword"
          }
        }
      }
    ],
    "properties": {
      "title": {
        "type": "text",
        "fields": {
          "keyword": {
            "type": "keyword",
            "ignore_above": 256
          }
        }
      },
      "content_text": {
        "type": "text"
      },
      "price": {
        "type": "double"
      },
      "stock": {
        "type": "integer"
      },
      "is_available": {
        "type": "boolean"
      },
      "created_at": {
        "type": "date",
        "format": "strict_date_optional_time||epoch_millis"
      }
    }
  }
}'
```

查看与验证 Mapping

```
# 查看完整映射
curl -u "$ES_USER:$ES_PASS" "$ES_URL/mapping_demo/_mapping?pretty"
```

```
# 查看特定字段映射
curl -u "$ES_USER:$ES_PASS" "$ES_URL/mapping_demo/_mapping/field/title?pretty"

# 查看字段统计信息
curl -u "$ES_USER:$ES_PASS" "$ES_URL/mapping_demo/_field_usage_stats?pretty"
```

测试动态模板效果

```
# 写入测试文档
curl -u "$ES_USER:$ES_PASS" -X POST "$ES_URL/mapping_demo/_doc/1" \
  -H "Content-Type: application/json" \
  -d '{
    "title": "Elasticsearch Mapping 最佳实践",
    "content_text": " 本文详细介绍 ES 的字段类型与动态映射控制...",
    "price": 99.99,
    "stock": 100,
    "is_available": true,
    "created_at": "2024-03-27T10:30:00Z",
    "user_id": "U10086",
    "product_id": "PROD-2024-001",
    "category_id": "CAT-TECH-001"
  }'

# 验证 user_id、product_id、category_id 被映射为 keyword
curl -u "$ES_USER:$ES_PASS" "$ES_URL/mapping_demo/_mapping?pretty" | \
  grep -A 3 '"user_id"\|"product_id"\|"category_id"'
```

典型查询测试

```
# 全文搜索
curl -u "$ES_USER:$ES_PASS" -X GET "$ES_URL/mapping_demo/_search?pretty" \
  -H "Content-Type: application/json" \
  -d '{
    "query": {
      "match": {
        "title": "Elasticsearch 最佳实践"
      }
    }
  }'

# 精确过滤 + 范围查询 + 排序
curl -u "$ES_USER:$ES_PASS" -X GET "$ES_URL/mapping_demo/_search?pretty" \
  -H "Content-Type: application/json" \
  -d '{
    "query": {
      "bool": {
        "must": [
          { "term": { "is_available": true } },
          { "range": { "price": { "gte": 50, "lte": 200 } } }
        ]
      }
```

```
  },
  "sort": [
    { "created_at": "desc" }
  ],
  "aggs": {
    "price_stats": {
      "stats": { "field": "price" }
    },
    "availability": {
      "terms": { "field": "is_available" }
    }
  }
}'
```

9.2 Java SDK 示例

Elasticsearch Java API Client 8.x 风格

```java
import co.elastic.clients.elasticsearch.ElasticsearchClient;
import co.elastic.clients.elasticsearch.indices.CreateIndexRequest;
import co.elastic.clients.elasticsearch.indices.GetMappingRequest;
import co.elastic.clients.elasticsearch.indices.GetMappingResponse;
import co.elastic.clients.json.jackson.JacksonJsonpMapper;
import co.elastic.clients.transport.ElasticsearchTransport;
import co.elastic.clients.transport.rest_client.RestClientTransport;
import org.apache.http.HttpHost;
import org.elasticsearch.client.RestClient;

import java.io.IOException;

public class MappingDemo {

    public static void main(String[] args) throws IOException {
        // 初始化客户端
        RestClient restClient = RestClient.builder(
            new HttpHost("localhost", 9200)).build();
        ElasticsearchTransport transport = new RestClientTransport(
            restClient, new JacksonJsonpMapper());
        ElasticsearchClient client = new ElasticsearchClient(transport);

        // 创建带动态模板的索引
        CreateIndexRequest createRequest = CreateIndexRequest.of(b -> b
            .index("orders")
            .settings(s -> s
                .numberOfShards("5")
                .numberOfReplicas("1")
            )
            .mappings(m -> m
                .dynamic(org.elasticsearch.xcontent.DynamicMapping.Strict)
                .dynamicTemplates(dt -> dt
                    .add(t -> t
```

```
                    .name("ids_as_keywords")
                    .template(tmpl -> tmpl
                        .match("*_id")
                        .mapping(mm -> mm
                            .keyword(k -> k.ignoreAbove(64))
                        )
                    )
                )
                .add(t -> t
                    .name("flags_as_booleans")
                    .template(tmpl -> tmpl
                        .match("is_*")
                        .mapping(mm -> mm.boolean_(b2 -> b2))
                    )
                )
                .add(t -> t
                    .name("amounts_as_scaled")
                    .template(tmpl -> tmpl
                        .match("amount_*")
                        .mapping(mm -> mm
                            .scaledFloat(sf -> sf.scalingFactor(100.0))
                        )
                    )
                )
            )
            .properties("order_id", p -> p.keyword(k -> k))
            .properties("product_name", p -> p
                .text(t -> t
                    .analyzer("ik_max_word")
                    .fields("keyword", f -> f
                        .keyword(k -> k.ignoreAbove(256))
                    )
                )
            )
            .properties("order_items", p -> p
                .nested(n -> n
                    .properties("sku_id", p2 -> p2.keyword(k -> k))
                    .properties("quantity", p2 -> p2.integer(i -> i))
                    .properties("unit_price", p2 -> p2
                        .scaledFloat(sf -> sf.scalingFactor(100.0))
                    )
                )
            )
            .properties("ext_attrs", p -> p.flattened(f -> f))
        )
    );

    client.indices().create(createRequest);
    System.out.println("Index 'orders' created successfully");

    // 获取并打印 Mapping
```

```
        GetMappingRequest getRequest = GetMappingRequest.of(g -> g
            .index("orders")
        );

        GetMappingResponse getResponse = client.indices().getMapping(getRequest);
        getResponse.result().forEach((indexName, mappingMetadata) -> {
            System.out.println("\n=== Index: " + indexName + " ===");
            System.out.println(mappingMetadata.sourceAsMap());
        });

        client.close();
    }
}
```

索引创建后的查询示例

```
// 嵌套查询：查找购买特定 SKU 且单价超过阈值的订单
SearchResponse<Order> searchResponse = client.search(s -> s
    .index("orders")
    .query(q -> q
        .nested(n -> n
            .path("order_items")
            .query(nq -> nq
                .bool(b -> b
                    .must(m -> m
                        .term(t -> t
                            .field("order_items.sku_id")
                            .value("SKU-IPHONE15P-256-DEEP")
                        )
                    )
                    .must(m -> m
                        .range(r -> r
                            .field("order_items.unit_price")
                            .gte(JsonData.of(5000))
                        )
                    )
                )
            )
            .innerHits(ih -> ih.name("matched_items"))
        )
    ),
    Order.class
);

// 处理结果
searchResponse.hits().hits().forEach(hit -> {
    System.out.println("Order ID: " + hit.source().orderId());
    hit.innerHits().get("matched_items").hits().hits().forEach(innerHit -> {
        System.out.println("  Matched item: " + innerHit.source());
    });
});
```

本章核心要义

Mapping 是 Elasticsearch 数据架构的基石，**字段类型的选择直接划定查询能力的边界**。`text` 与 `keyword` 的本质差异源于分词机制的有无，`nested` 与 `object` 的核心区分在于数组元素关联性的保持，`flattened` 是动态键值对场景的映射爆炸防线。生产环境必须坚持**“显式优于隐式、约束优于放任”**的原则，通过动态模板、索引模板、三层防御体系等机制，在灵活性与可控性之间取得平衡。**前期充分的建模投入，是避免后期高昂重建代价的最佳投资**。